Week 11: Foundation Models and more, plus revision

Machine Learning and Advanced Analytics — Session XXII

This session is mainly a **review session** with no new practical, but the uploaded materials include two important things: final comments on foundation models/neural networks, and the exam revision guidance. The revision deck says the exam has **two sections**: Section A is scenario-based multiple choice worth 50 marks and weighted at 1.5x, while Section B is code review and understanding worth 25 marks.

---

1. What Week 11 is about

Week 11 is not introducing a completely new algorithm. It is about consolidating the full module:

- Real-world model use and data challenges
- Feature importance and feature selection
- SVMs
- Time series and temporal evaluation
- Neural networks
- Foundation models
- Causal inference
- Code understanding and debugging

The revision slide explicitly lists these as the main areas covered across the module, including missing data, pipelines, unbalanced classes, learning curves, temporal baselines, SVMs, variable importance, PDP, SHAP, causal inference, neural networks, and foundation models.

---

2. Foundation models

A **foundation model** is a pre-trained deep neural network that acts as a backbone for many downstream tasks.

In simple words:

> *A foundation model is trained once on a huge and diverse dataset, then adapted or reused for many different tasks.*

Examples:

Input type	Foundation model area
Text	Large language models
Images	Vision models / multimodal models

Input type	Foundation model area
Time series	Time series foundation models
Speech	Speech recognition / speech generation models
Structural data	Graph or structured-data models

The lecture defines foundation models as pre-trained deep neural networks that are adaptable across multiple tasks and trained on massive, diverse datasets to capture general features across domains.

---

3. Why foundation models are useful

Foundation models are useful because they reduce the need to train a new model from scratch.

Instead of collecting a huge labelled dataset and training a bespoke model, we can start with a model that already learned useful general representations.

Benefits:

Benefit	Meaning
Reduced development time	Less time spent training from zero
Reduced data requirement	Can work with less task-specific data
Better representations	Learns useful features from massive data
Adaptability	Can be reused across different tasks
Potentially better performance	May outperform smaller custom models

Important caveat:

> *Foundation models can improve performance, but not always. They still need to be chosen and adapted appropriately.*

The slides explicitly state that foundation models can reduce development time and resources, and may potentially improve performance, but not always.

---

4. Foundation models are not fully generic

A foundation model is **general**, but not magically suitable for every possible problem.

The lecture notes explain that foundation models are general because they use generic inputs and outputs or internal representations that can support more than one task. However, there are still different foundation models for different input data, such as text, images, and time series.

Example:

Problem	Suitable foundation model
Answering questions from documents	LLM
Image captioning	Vision-language model
Object recognition	Vision foundation model
Forecasting sales	Time-series model
Medical Q&A	General LLM adapted to medical domain

Exam phrase:

> *Foundation models are general-purpose within a broad data/task family, not fully generic for every possible problem.*

---

5. Embeddings and representations

Foundation models are mainly about learning **good representations**.

An **embedding** is a numerical representation of an input.

For example:

- A word becomes a vector
- A sentence becomes a vector
- An image becomes a vector
- A customer sequence becomes a vector

The slide says embeddings are outputs of internal layers in a neural-network context, and foundation models are often a mixture of representation learning and output modelling.

Simple explanation:

> *Instead of manually designing features, the model learns useful internal features automatically.*

Example:

For text, an embedding may place similar words or sentences close together in vector space.

For images, an embedding may represent visual features such as edges, shapes, objects, and style.

---

6. Ways to adapt foundation models

The lecture covers several ways to adapt or use foundation models.

6.1 Prompt engineering

Prompt engineering is the lightest form of adaptation.

The model itself is not changed. We only change the input prompt.

Example:

> *"Classify the following text as sports, politics, or entertainment: [input text]"*

This can turn a general LLM into a simple classifier without training a new model.

Good for:

- Quick prototypes
- Simple classification
- Copywriting variations
- One-shot code generation
- Demos

Weakness:

- Less reliable for production
- Sensitive to wording
- Does not permanently teach the model new knowledge

The lecture describes prompt engineering as a very lightweight adaptation where fixed context is provided for a general problem.

6.2 RAG: Retrieval-Augmented Generation

RAG means **Retrieval-Augmented Generation**.

The model is not changed. Instead, relevant information is retrieved from a database or knowledge base and passed to the LLM as extra context.

Process:

1. User asks a question
2. System searches a document/database
3. Relevant information is retrieved
4. The LLM receives both the question and retrieved context
5. The LLM generates an answer

Simple definition:

> *RAG connects an LLM to external information so it can answer using retrieved context without retraining.*

Good for:

- Company knowledge bases
- Customer support bots

- Document Q&A
- Reducing hallucination
- Keeping information up to date

Weakness:

- Depends on retrieval quality
- Bad retrieved context can produce bad answers
- Does not actually change model weights

The lecture explains that in RAG, the LLM is not changed; new data is indexed in a database and retrieved information is passed with the question as extra context.

6.3 Context engineering

Context engineering is broader than prompt engineering.

Prompt engineering focuses on wording the immediate prompt.

Context engineering focuses on designing the full environment in which the model operates.

This may include:

- System instructions
- Chat history
- Tools
- Memory
- Retrieved documents
- Codebases
- Structured output requirements
- Project rules
- User preferences

The lecture frames context engineering as a broader system-level choice, especially relevant to agents, memory, tools, codebases, and retrieved information.

Exam phrase:

> *Prompt engineering is about what to say to the model now; context engineering is about designing the whole information environment the model uses.*

6.4 Fine-tuning

Fine-tuning means taking a pre-trained model and training it further on a smaller, task-specific dataset.

Example:

A general LLM can be fine-tuned on medical text, legal documents, company emails, or labelled customer-service responses.

The slides explain that during fine-tuning, some layer weights are unfrozen and standard neural-network training is performed on new data. The model adapts to the new use case, but can also overfit.

Benefits:

Benefit	Explanation
More task-specific	Learns domain/task patterns
More consistent	Can improve output style or behaviour
Powerful adaptation	Changes the model itself

Risks:

Risk	Explanation
Overfitting	Model adapts too much to small data
Cost	More expensive than prompting/RAG
Data requirement	Needs suitable training examples
Maintenance	Needs retraining if data changes

6.5 Parameter-efficient tuning

Parameter-efficient tuning adapts a model by changing only a small subset of parameters.

Instead of retraining the whole model, methods such as adapter layers can be inserted between existing layers.

The lecture describes this as modifying a small subset of model parameters, for example using adapter layers, to improve performance without full retraining.

Simple explanation:

> *Parameter-efficient tuning is a cheaper middle ground between prompt engineering and full fine-tuning.*

6.6 Reinforcement Learning from Human Feedback, RLHF

RLHF is used to align model outputs with human preferences.

General process:

1. A base model generates responses
2. Humans compare or rate responses
3. A smaller preference/reward model learns what humans prefer

4. The base model is further trained to produce preferred outputs

The lecture notes describe RLHF as using a standard model and building a smaller model from human feedback, with humans still playing an important role in making ChatGPT-style systems work.

Simple explanation:

> *RLHF teaches a model not just to predict text, but to produce outputs humans prefer.*

---

7. Prompting vs RAG vs fine-tuning

Method	Changes model weights?	Adds new knowledge?	Best use
Prompt engineering	No	Only in prompt	Quick tasks, demos
RAG	No	Yes, through retrieval	Document Q&A, current/company knowledge
Context engineering	No, usually	Yes, through system design	Agents, production LLM systems
Fine-tuning	Yes	Yes, through training	Domain/task adaptation
Parameter-efficient tuning	Partly	Yes	Cheaper task adaptation
RLHF	Yes	Behaviour alignment	Making outputs more helpful/preferred

Exam-ready distinction:

> *Use RAG when the problem is mainly missing or changing knowledge. Use fine-tuning when the problem is model behaviour, style, task format, or domain-specific patterns that need to be internalised.*

---

8. Reasoning LLMs and chain of thought

The lecture briefly mentions reasoning LLMs and chain-of-thought-style reasoning.

Standard LLMs are described as fast and associative: they generate tokens using pre-trained knowledge, but may have limited planning ability and can hallucinate logic.

Reasoning LLMs are described as slower and more deliberate: they may check steps, decompose problems, and use verification loops.

Important exam-safe explanation:

> *Reasoning LLMs aim to improve performance on complex tasks by decomposing problems into steps, checking intermediate reasoning, and searching or verifying possible answers.*

Use cases:

- Maths
- Coding
- Multi-step planning
- Complex question answering
- Logical reasoning

---

9. Verifiers and reward models

The lecture mentions two types of verifiers:

Verifier	Meaning
Outcome Reward Model, ORM	Judges only the final answer
Process Reward Model, PRM	Judges the reasoning process or intermediate steps

The idea is that reasoning models can generate multiple possible answers or solution paths, then use a verifier or voting method to choose the best one.

Simple explanation:

> *A verifier helps the model choose better answers by scoring either the final result or the process used to reach it.*

---

10. Beyond transformers: MoE and Mamba

The final neural-network slides briefly mention newer architectures such as **Mixture of Experts** and **Mamba**.

You do not need deep technical details, but conceptually:

Mixture of Experts, MoE

MoE uses different “expert” subnetworks and activates the most relevant ones for a given input.

Main idea:

> *Use the right expert at the right time to reduce computation.*

Mamba / state-space style models

Mamba-style models aim to handle long contexts more efficiently than standard transformers.

The lecture notes suggest these models are trying to find a better trade-off: transformers avoid compressing history but are computationally expensive, while older recurrent models compress history but lose information.

Exam-safe phrase:

> *Newer architectures try to reduce the computational cost of attention while retaining useful long-context information.*

---

11. Revision: exam structure

From the revision slides:

Section	Marks	Style
Section A	50 marks, weighted 1.5x	Scenario-based multiple choice
Section B	25 marks	Code review and understanding

The revision deck says to answer all questions, answer the ones you can do quickly first, and return to harder ones later.

---

12. What Section A may include

The revision deck gives an indicative breakdown for Section A:

Topic	Approx. marks
Feature importance and selection	15
Real-world model use, model complexity, data challenges	12
Causal inference	7
Time series	4
Neural networks	12

The slide also warns that topics may be mixed, so an SVM question may appear inside another topic area.

---

13. What Section B may include

Section B is about **understanding the process**.

You may be shown code and asked to identify issues, explain why they matter, and suggest fixes.

Likely issues:

- Data leakage
- Wrong train/test split
- Bad pipeline structure
- Scaling outside cross-validation
- Imputation before splitting
- Wrong evaluation metric
- Ignoring class imbalance
- Temporal leakage
- Poor hyperparameter tuning
- Procedural overfitting
- Misinterpreting model results
- Incorrect feature construction

The revision slide shows Section B as code review and asks students to identify best-practice violations, explain consequences, and suggest corrections.

---

14. What will not be examined

The revision slides say you will not be asked to write proofs or maths equations.

For SVMs, for example, the formal optimisation problem and loss function formulas are not examined. However, you still need to understand what the maths implies conceptually, such as how gradient descent is used and why vanishing gradients are a problem.

Important:

> *Do not spend your last revision time memorising formulas. Focus on concepts, scenario interpretation, and code/process understanding.*

---

15. High-priority revision topics

15.1 Missing data

Know the difference between:

Type	Meaning
MCAR	Missing completely at random
MAR	Missing at random conditional on observed variables
MNAR	Missing not at random; missingness depends on unobserved value

Know how to deal with missing data:

- Drop rows/columns only when justified
- Mean/median imputation for numeric features
- Most frequent/category imputation for categorical features
- Missingness indicator if missingness itself may be informative
- Use pipelines to avoid leakage

Exam point:

> *Imputation should be learned on training data only, then applied to validation/test data.*

15.2 Categorical encoding

Know when to use:

Encoding	Use
One-hot encoding	Nominal categories
Ordinal encoding	Ordered categories
Target encoding	High-cardinality categories, but leakage risk
Binary indicators	Yes/no features

Exam point:

> *Encoding must be fitted only on training data inside a pipeline, especially during cross-validation.*

15.3 Data leakage

Data leakage happens when information from outside the training data is used during model training.

Examples:

- Scaling before train/test split
- Imputing before train/test split
- Feature selection before cross-validation
- Using future data in time-series features
- Using variables only known after the prediction time
- Tuning repeatedly on the test set

Exam phrase:

> *Leakage makes validation/test performance look better than it will be in the real world.*

15.4 Imbalanced classes

In imbalanced classification, accuracy can be misleading.

Example:

If 95% of customers do not churn, a model predicting “no churn” for everyone gets 95% accuracy but is useless.

Better metrics:

- Precision
- Recall
- F1-score
- ROC-AUC
- PR-AUC
- Confusion matrix

Possible fixes:

- Class weights
- Resampling
- Threshold adjustment
- Better metric selection

Exam phrase:

> *For rare positive classes, PR-AUC and recall/precision trade-offs may be more informative than accuracy.*

15.5 Overfitting

Traditional overfitting:

> *Model performs well on training data but poorly on unseen data.*

Signs:

Training error	Validation/test error	Meaning
Low	High	Overfitting
High	High	Underfitting
Low	Low	Good fit

Fixes:

- Simpler model
- Regularisation

- More data
- Early stopping
- Dropout for neural networks
- Better feature selection
- Cross-validation

15.6 Procedural overfitting

Procedural overfitting happens when the modelling process is repeatedly adjusted to perform well on validation/test data.

Example:

Trying many models and feature sets, repeatedly checking the same test set, then reporting the best result.

Exam phrase:

> *Procedural overfitting means overfitting to the evaluation process rather than only overfitting model parameters.*

Fix:

- Keep a final untouched test set
- Use cross-validation for model selection
- Report honest final performance once

---

16. Feature importance and model understanding

Know the difference between:

Concept	Meaning
Feature importance	Which features matter most to a model
Feature selection	Removing less useful features
Feature extraction	Creating new features/representations
Feature understanding	Interpreting what features mean in context

Feature importance can help:

- Simplify the model
- Reduce data collection cost
- Improve interpretability
- Understand business drivers

- Identify possible interventions

But be careful:

> *Feature importance does not prove causality.*

A feature may be predictive but not causal.

---

17. PDP and SHAP

Partial Dependence Plot, PDP

A PDP shows how the model prediction changes as one feature changes, averaging over other features.

Useful for:

- Understanding direction of model relationship
- Seeing non-linear effects
- Communicating model behaviour

Limitation:

- Can be misleading when features are strongly correlated

SHAP

SHAP explains individual predictions by estimating how each feature contributes to the prediction.

Useful for:

- Local explanation
- Individual-level interpretation
- Understanding why a specific prediction was made

Exam distinction:

Method	Best for
Feature importance	Overall importance
PDP	Average effect of a feature on predictions
SHAP	Individual prediction explanation

---

18. Time-series revision

Time-series problems require special care because data is ordered.

Important issues:

Issue	Explanation
Temporal leakage	Using future information to predict the past
Lagged features	Features created from previous time periods
Rolling/tumbling windows	Aggregates over fixed historical windows
Expanding windows	Aggregates from start up to current time
Temporal validation	Train on past, validate/test on future
Temporal baselines	Compare against simple time-aware baselines

Exam phrase:

> *Random train/test splits are often inappropriate for time-series data because they can leak future information into training.*

---

19. SVM revision

Know conceptually:

Concept	Meaning
Margin	Separation between classes
Support vectors	Points closest to the boundary
Kernel	Allows non-linear decision boundaries
Primal form	Often useful when features are fewer
Dual form	Useful for kernels and certain high-dimensional settings
Scaling	SVMs usually require feature scaling

What to change:

Problem	Possible adjustment
Underfitting	Increase complexity, adjust kernel/C
Overfitting	Reduce C, simpler kernel
Poor scaling	Standardise features
Non-linear boundary	Use kernel

No need to memorise formal optimisation maths, but understand what the concepts imply.

---

20. Neural-network revision

The revision deck says neural networks are important conceptually, especially why they are hard to train, vanishing/exploding gradients, activation functions, initialisation, optimisers, complexity control, dropout, regularisation, and batch normalisation.

Key neural-network points

Topic	Know this
Activation functions	Add non-linearity
Gradient descent	Updates weights to reduce loss
Learning rate	Controls update size
Vanishing gradients	Gradients become too small in deep networks
Exploding gradients	Gradients become too large/unstable
Initialisation	Helps gradients flow properly
Dropout	Randomly disables neurons during training
Regularisation	Reduces overfitting
Batch normalisation	Stabilises training by normalising activations
Adam	Adaptive optimiser using momentum and adaptive learning rates
AdaGrad	Adapts learning rates based on past gradients

---

21. Causal inference revision

The revision deck highlights:

- Counterfactuals
- ATE vs ATT
- Simpson's paradox
- DAGs
- Conditional dependencies
- Minimum adjustment set
- Refutation tests

Key definitions

Concept	Meaning
Counterfactual	What would have happened under another treatment
ATE	Average treatment effect across the population
ATT	Average treatment effect on treated units
Confounder	Affects both treatment and outcome
DAG	Directed graph encoding causal assumptions
Refutation	Robustness test because true causal ground truth is unavailable

The revision deck states that DAGs encode domain knowledge and help identify the minimum set of variables to condition or stratify on.

---

22. Simpson's paradox revision example

In the revision deck, the algorithm example shows:

Group	Old algorithm A	New algorithm B
Low-income visitors	10/400 = 2.5%	4/200 = 2.0%
High-income visitors	40/600 = 6.6%	50/800 = 6.2%
Overall	50/1000 = 5.0%	54/1000 = 5.4%

Overall, B looks better.

But within each subgroup, B is worse.

Conclusion:

> *The new algorithm is not better; the apparent improvement comes from a different composition of visitors.*

The revision slide says the fix is to sample in a way that represents the population, such as stratified sampling, but this requires knowing what variables to stratify on, connecting to the unconfoundedness assumption.

---

23. Refutation tests in causal inference

Because causal inference usually lacks ground truth, we test whether the estimate is robust.

The revision deck gives examples:

Random common cause refutation

Add a random variable that should not cause the outcome.

Expected result:

> *The causal estimate should not change much.*

Placebo treatment refutation

Randomise the treatment.

Expected result:

> *No real causal effect should be detected.*

If the model still finds an effect, something is wrong.

The revision deck explains that refutations are used because there is no ground truth, so we test robustness against assumptions.

---

24. Final exam strategy

Use this order:

1. Do quick Section A scenario questions first
2. Skip confusing ones and return later
3. For Section B, read the code for leakage first
4. Check whether preprocessing is inside a pipeline
5. Check train/validation/test split logic
6. Check whether the metric matches the problem
7. For temporal data, check whether future data leaked
8. For causal questions, look for confounders, counterfactuals, ATE/ATT, and DAG assumptions
9. Do not waste time writing formulas unless needed conceptually

The revision slide explicitly recommends answering the questions you can answer quickly first and coming back to the others.

---

25. One-page memory summary

Week 11 is mainly revision. Foundation models are pre-trained deep neural networks trained on massive diverse datasets and reused across downstream tasks. They work by learning useful representations or embeddings. They can be adapted through prompt engineering, RAG, context engineering, fine-tuning, parameter-efficient tuning, and RLHF. Prompting changes only the input, RAG adds retrieved knowledge without changing the model, and fine-tuning changes the model weights. Context engineering is broader system design around the model. Reasoning LLMs aim to improve complex problem solving through decomposition, verification, and

deliberate reasoning. Newer architectures such as MoE and Mamba aim to improve efficiency and long-context handling.

For the exam, focus on concepts, scenarios, and code understanding. Section A is scenario-based multiple choice; Section B is code review. High-priority topics include missing data, categorical encoding, leakage, imbalanced classes, overfitting, procedural overfitting, learning curves, time-series evaluation, feature importance, PDP, SHAP, SVMs, neural networks, causal inference, DAGs, ATE/ATT, Simpson’s paradox, and refutations. Proofs and formulas are not examined, but you must understand what the concepts mean in practice.